CS6868 Research Mini-Project

Herlihy's Universal Construction

Venkat Koundinya Singaraju
cs23b060@smail.iitm.ac.in

Ventrapragada Krishna Sameer
cs23b078@smail.iitm.ac.in

Siddharth S George
cs23b098@smail.iitm.ac.in

May 1, 2026

Abstract

The main goal of this project is to understand the performance cost of lock-free and wait-free universal constructions. These constructions are used to make any sequential object linearizable. To understand this, we first implement the algorithms for lock-free and wait-free universal constructions. We linearize a few sequential objects, such as Stack, Queue, Binary Search Tree, Sorted Tree, and Skip List, using universal constructions. We then benchmark the universal constructions of these objects against the purpose-built lock-free data structures. The lock-free universal construction is faster than the wait-free universal construction, albeit by a very small margin. Both the lock-free and the wait-free universal constructions are much slower than purpose-built implementations (by orders of magnitude). Another goal of this project is to understand how frequently threads help others vs. execute their own operations in the wait-free universal constructions. It turns out that the threads almost always help other threads and execute their operations very rarely.

1 Goals

The goals of this project are to understand the performance impact of universal constructions and to measure the cost of helping and the performance impact of the helping mechanism in the wait-free universal construction.

The following data structures and algorithms were implemented as part of the project:

- Lock-free universal construction algorithm
- Wait-free universal construction algorithm
- Consensus object using Compare and Set
- Lock-free Treiber Stack
- Michael-Scott Queue
- Mutex-protected sequential Stack and Queue
- Sequential Binary Search Tree
- Sequential Sorted List
- Sequential Skip List
- Sequential Stack

- Sequential Queue
- Lock-free and Wait-free versions of the above five Sequential Objects using the universal construction algorithms

The above project is very interesting because we have learned to linearize any sequential object. We also got to explore the performance cost of the universal constructions.

The above project connects to Lectures 1,2,3,7 and 8 of the course. Lectures 1,2, and 3 deal with linearizability, lock-freedom, and wait-freedom, while Lectures 7 and 8 deal with data structures related to this project.

In short, the following is the goal of the project: What is the practical performance cost of generality — how much throughput do the universal constructions sacrifice compared to purpose-built lock-free data structures, and does the wait-free variant's helping mechanism introduce prohibitive overhead?

2 Background [2] [3]

2.1 Compare and Set consensus

A consensus object provides a single method called `decide`. This method is called by multiple threads with their respective inputs at most once. The method returns a value that is consistent(all threads return the same value) and valid(the common value is the input of some thread).

A class C solves the consensus of n threads if there exists an implementation of the consensus object for n threads using any number of objects in class C and any number of atomic registers. The consensus number of class C is the largest n for which that class solves the consensus of n-threads.

In a system of n or more concurrent threads, it is impossible to implement a wait-free or lock-free object with consensus number n from objects belonging to a class with a lower consensus number. (Proof omitted)

A register providing `compareAndSet` and `get` methods has an infinite consensus number (Proof omitted). Thus, we chose this to implement our consensus object.

Listing 1: CAS Consensus

```
type 'a t = {
  num_threads : int;
  proposed : 'a option Atomic.t array;
  winner : int Atomic.t;
}

let create num_threads = {
  num_threads;
  proposed = Array.init num_threads (fun _ -> Atomic.make None);
  winner = Atomic.make (-1);
}

let propose consensus value tid =
  Atomic.set (consensus.proposed.(tid)) (Some value)

let decide consensus value tid =
  propose consensus value tid;
  if Atomic.compare_and_set consensus.winner (-1) tid then begin
    match Atomic.get consensus.proposed.(tid) with
    | Some v -> v
    | None -> failwith "This should never happen 1"
```

```
  end
  else begin
    let winner_tid = Atomic.get consensus.winner in
    match Atomic.get consensus.proposed.(winner_tid) with
    | Some v -> v
    | None -> failwith "This should never happen 2"
  end
```

2.2 Lock-free Universal Construction

A class C is universal if one can construct a wait-free implementation of any object from some number of objects of C and some number of read–write registers.

Listing 2: Node Class

```
type ('a,'b) t =
  {
    decide_next : ('a,'b) t CASConsensus.t;
    next : ('a,'b) t option Atomic.t;
    seq : int Atomic.t;
    invoc : ('a -> ('a * 'b)) option (*invoc is a function*)
  }

let create invoc num_threads = {
    decide_next = CASConsensus.create num_threads;
    next = Atomic.make None;
    seq = Atomic.make 0;
    invoc;
  }

let max arr =
  let rec aux acc i =
    if i >= Array.length arr then acc
    else
      let cur = arr.(i) in
      if Atomic.get cur.seq > Atomic.get acc.seq then aux cur (i+1)
      else aux acc (i+1)
  in
  aux arr.(0) 1
```

Listing 3: Lock-Free Universal Construction

```
type ('a,'b) t = {
  head : ('a,'b) Node.t Atomic.t array;
  tail : ('a,'b) Node.t;
  num_threads : int;
}

let create num_threads =
  let tail = Node.create None (num_threads+1) in
  Node.set_seq tail 1;
  {
    head = Array.init (num_threads+1) (fun _ -> Atomic.make tail);
    tail;
    num_threads=num_threads+1
  }

let apply lfu_obj new_obj invoc =
```

```
  let tid = Domain.self_index () in
  let prefer = Node.create (Some invoc) lfu_obj.num_threads in
  let rec aux () =
    if Node.get_seq prefer <> 0 then ()
    else begin let before = Node.max (Array.map Atomic.get lfu_obj.head) in
    let after = CASConsensus.decide (Node.get_decide_next before) prefer tid in
    Node.set_next before after;
    Node.set_seq after (Node.get_seq before + 1);
    Atomic.set lfu_obj.head.(tid) after;
    aux ()
    end
  in
  aux ();
  let rec app current acc =
    if match current with
      | Some node -> node == prefer
      | None -> false
    then begin
      match Node.get_invoc prefer with
        | Some invoc -> invoc acc
        | None -> failwith "This should never happen 3"
      end
    else begin
      match current with
      | Some node -> let new_acc = match Node.get_invoc node with
                                   | Some invoc -> fst (invoc acc)
                                   | None -> failwith "This should never happen 4"
                     in
                     app (Node.get_next node) new_acc
      | None -> failwith "This should never happen 5"
    end
  in
  app (Node.get_next lfu_obj.tail) new_obj
```

This construction transforms any sequential object into a lock-free linearizable concurrent object. This construction assumes that sequential objects are deterministic: If we apply a method to an object in a particular state, then there is only one possible response and one possible new object state.

An object is represented as a combination of a sequential object in its initial state and a linked list of nodes representing the sequence of operations performed on the object.

The function `apply` takes an invocation as an argument. `invoc` represents a call to a function to a sequential object. It takes an object, changes the object, and returns a tuple of the modified object and the return value.

A thread attempting to call `apply` creates a node to hold its invocation. The threads then compete to append their respective nodes to the head of the log by running an n-thread consensus protocol to agree which node was appended to the log with input as the threads' nodes.

Once a node is part of the log, the thread computes the response to its invocation by traversing the log from the tail to the newly added node. We locate the head of the log by maintaining an array in which each element in the array represents the head of the respective thread. The `max` method performs a collect, reading the array, and returning the node with the highest sequence number

2.3 Wait-free Universal Construction

The algorithm described in the above section is lock-free because a thread can keep losing the consensus and become starved. In the wait-free implementation, whenever a thread wants to

append a node, it announces this to all other nodes. This is done by maintaining an announce array. In the first loop, a thread checks if some other node wants to append itself. If there is one, this thread does it, otherwise, it appends its own node. This ensures every thread's node gets appended in a finite number of steps. It is wait-free because inside the loop, each thread checks `announce[(before.seq + 1)]` - this is a deterministic way to round robin over all thread-ids. If the selected thread's node is still unappended `(seq == 0)`, the current thread helps it by preferring that node in the consensus race instead of its own. This guarantees every announcement is checked at least once every n appends - so any stuck thread gets helped within at most n steps.

Listing 4: Wait-Free Universal Construction

```
let apply wfu_obj new_obj invoc =
  let tid = Domain.self_index () in
  let anc = Node.create (Some invoc) wfu_obj.num_threads in
  Atomic.set wfu_obj.announce.(tid) anc;
  Atomic.set wfu_obj.head.(tid) (Node.max (Array.map Atomic.get wfu_obj.head));
  let rec aux () =
    let ann_i = Atomic.get wfu_obj.announce.(tid) in
    if Node.get_seq ann_i <> 0 then ()
    else begin
      let before = Atomic.get wfu_obj.head.(tid) in
      let idx = ((Node.get_seq before) + 1) mod wfu_obj.num_threads in
      let help =  Atomic.get wfu_obj.announce.(idx) in
      let prefer = begin
        if Node.get_seq help = 0 then help
        else Atomic.get wfu_obj.announce.(tid)
      end in
      let after = CASConsensus.decide (Node.get_decide_next before) prefer tid in
      Node.set_next before after;
      Node.set_seq after (Node.get_seq before + 1);
      Atomic.set wfu_obj.head.(tid) after;
      aux ()
    end
  in aux ();
  let ann_i = Atomic.get wfu_obj.announce.(tid) in
  Atomic.set wfu_obj.head.(tid) ann_i;
  let rec app current acc =
    if match current with
      | Some node -> node == ann_i
      | None -> false
    then begin
      match (Node.get_invoc ann_i) with
      | Some invoc -> invoc acc
      | None -> failwith "This should never happen 6"
    end
    else begin match current with
    | Some node -> begin
      match (Node.get_invoc node) with
      | Some invoc -> let (next, _) = invoc acc in app (Node.get_next node) next
      | None -> failwith "This should never happen 7"
      end
    | None -> failwith "This should never happen 8"
    end
  in
  app (Node.get_next wfu_obj.tail) new_obj
```

3 Tasks Undertaken

3.1 Implementation

3.1.1 Universal constructions

We implemented the lock-free and the wait-free universal constructions. The head and announce arrays were atomic in the implementation of universal constructions. The proposed array is atomic in the Compare and Set Consensus object. The reference to the next node and the sequence number are atomic in the Node object. All of the above data structures have concurrent accesses that involve at least one write. The above implementations used only array. They do not require any other data structures. The following are links to the related files:

- Compare and Set Consensus object
- Node object
- Lock-Free universal construction
- Wait-Free universal construction
- Converts sequential objects into linearizable concurrent objects using the universal constructions

3.1.2 Sequential Objects

We also implemented Sequential Binary Search Tree, Sequential Queue, Sequential Skip List, Sequential Sorted List, and Sequential Stack. The above sequential objects are compatible with the format required by the universal constructions. These were then made lock-free linearizable and wait-free linearizable using the lock-free and wait-free universal constructions, respectively. Lists and Arrays were the only data structures used in the implementation of the above sequential objects. The following are links to the related files:

- Sequential Binary Search Tree
- Sequential Queue
- Sequential Skip List
- Sequential Sorted List
- Sequential Stack

3.1.3 Purpose-built object implementations

We also took a lock-free Treiber stack and a Michael-Scott queue from Lecture 8. We implemented the Mutex-protected sequential versions of Stack and Queue. The Mutex protected Stack used a list underneath and the Mutex protected Queue used a Queue underneath. The following are links to the related files:

- Mutex protected Queue
- Mutex protected Stack
- Michael-Scott queue
- Lock-free Treiber stack

3.1.4 Benchmarking

We measured the cost of helping of Wait-free universal construction using an instrumented version of Wait-free universal construction. It is almost the same as the original Wait-free universal construction, but it keeps track of the number of times a thread helps another thread vs. the number of times it executes its own operations. The following are links to related files:

- Compares the Mutex protected version, purpose-build lock free version, Universally constructed lock free version, and Universally constructed wait free version of stack and queue
- Measures cost of helping for Universally constructed wait-free versions of Sequential Objects
- Compares the throughput of wait-free universal construction of Sequential Objects with the lock-free universal construction of Sequential Objects

3.1.5 Design choices

We faced two main design choices while working on this project:

- Deciding on the type of function `invoc`:
  We had originally designed it in such a way that it takes an object as input and returns the modified object. It works, but it is slightly inconvenient as it would get tough to find the return value of the function.
- Accessing Domain index:
  We initially thought of passing the tid explicitly since `Domain.self ()` did not guaranty that it would return a number in the range [0 to n]. This caused issues since we would not guaranty unique tid's when making Lin and STM tests. We then chose to use a hashtable to map `Domain.self ()` to a unique number, but this caused data races on the map when we ran TSAN. We finally decided to use `Domain.self_index ()` which was not in the documentation of the Domain module (some other group suggested it), which worked out in the end.

3.2 Testing and verification

Some of the bugs that the tests caught were the following:

- Why tid should be unique:
  While running QCheck-Lin on the first implementation with tid as a parameter, the stm-tests worked fine. However, the Lin-tests flagged an issue. We had been using `int_small` to generate numbers for the tid and taking `mod n`. The error was caused because two threads were using the same tid. The error helped us find an overlap and understand why the tids must be unique.
- Finding Data races using TSAN:
  After changing the implementation to a hashtable implementation, both the QCheck tests passed. However, TSAN flagged the data race while accessing the hashtable. This helped us shift the implementation to a race free one.
- Other manual tests:
  These did not help fix the constructions per se. They helped in finding errors in our sequential objects that we were testing.

4 Evaluation

We used a 16-core CPU. We have used up to 15 threads for our benchmarking. We ran 100 iterations, averaged the results, and plotted them. We ran it multiple times to cancel out the noise.

The lock free and wait free implementations have almost similar performance for stack and queue, while they are much slower than the Mutex-protected version and purpose-built lock free version(differ by orders of magnitude). This is because every time a method is called, all the previous methods are also executed in a sequence starting from the beginning of the list in the universal constructions. The purpose-built lock-free version has the best performance for both stack and queue, as shown in Figures 1 and 2.

For Binary Search Tree, lock-free performs better than wait-free (although only by a small margin). For all other data structures, the performance seems to be roughly the same as shown in Figures 3, 4,and 5. Thus, from these figures, we can conclude that the two universal constructions have similar performance with the lock-free implementation having a slight edge.

The results did not match our prior expectations, as we expected the lock-free universal construction to outperform the wait-free universal construction by a good margin, since there is the overhead of helping mechanism in wait-free universal construction. Later, on analysing the code, we found that this overhead is offset by repeated calls to `max` function in lock-free universal construction. The function is called in every iteration of the lock-free universal construction, but it is called only once in the wait-free universal construction. Everytime the `max` function is called, it iterates through the entire head array and finds the maximum sequence number; hence it has a significant performance overhead. Another reason is because both the lock-free and wait-free variants are bottlenecked with the history replay cost. This dominates, and hence the difference between the lock-free and the wait-free variants is swamped.

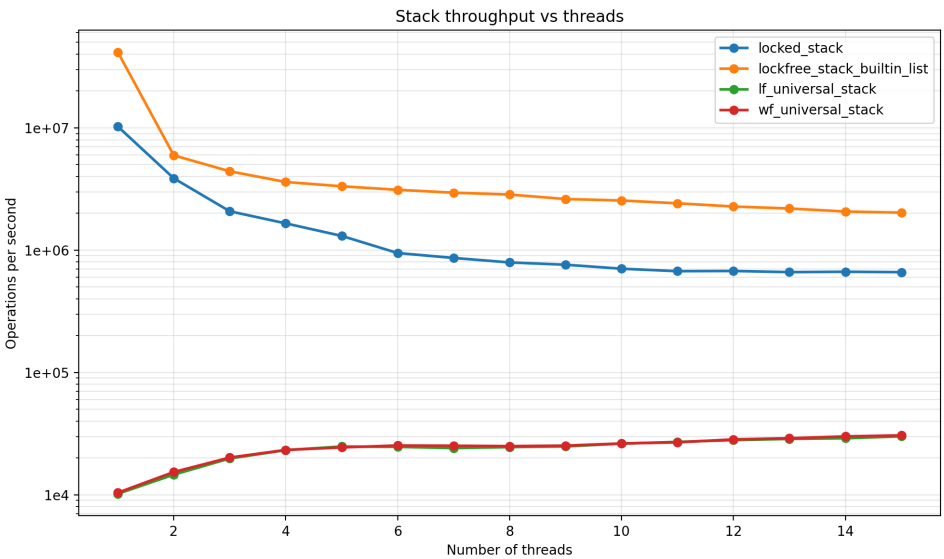

Figure 1: Throughput vs. threads for different implementations of stack.

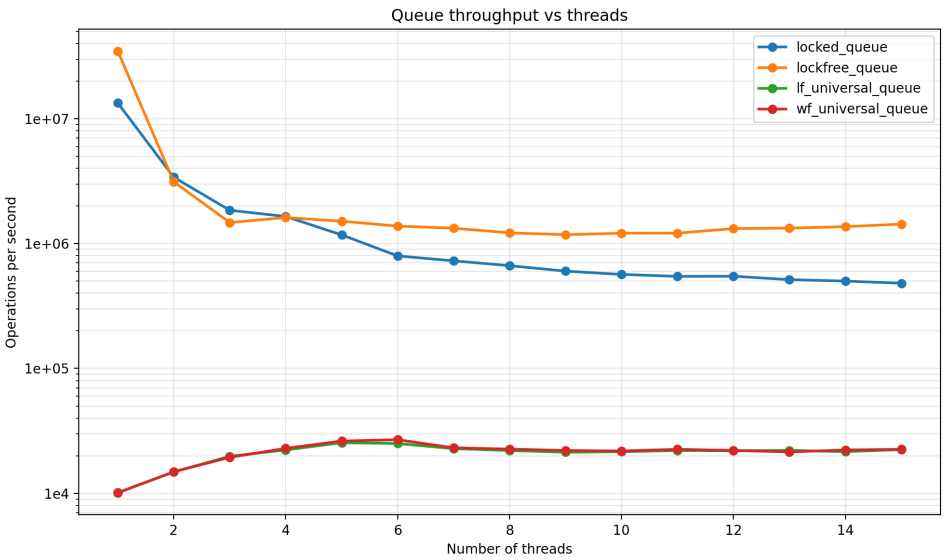

Figure 2: Throughput vs. threads for different implementations of queue.

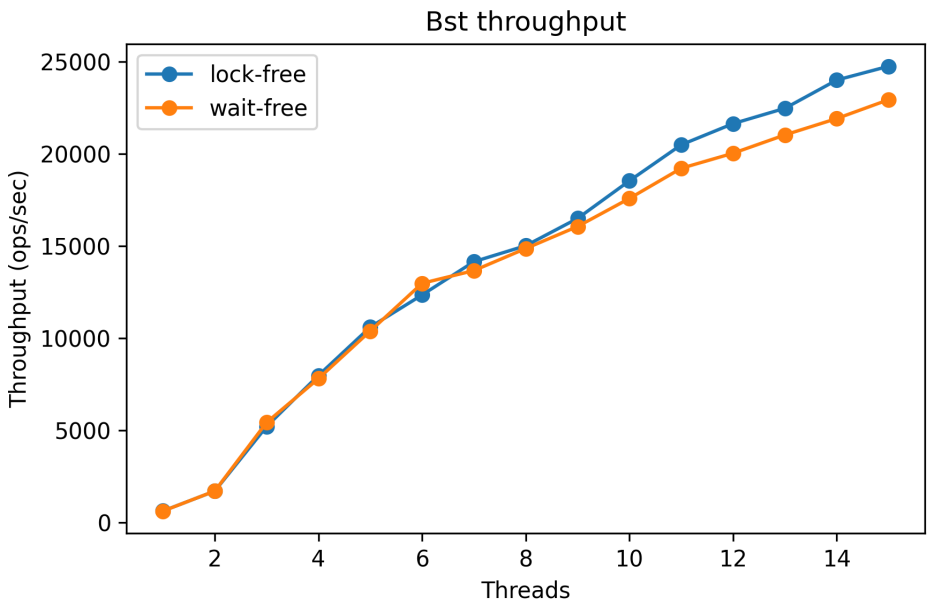

Figure 3: Wait-free vs Lock-free comparison for BST

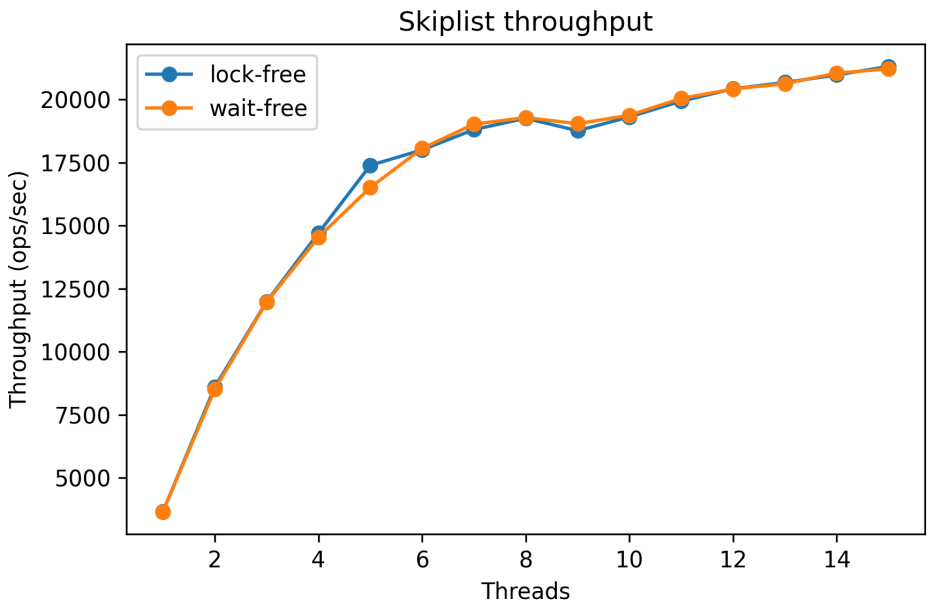

Figure 4: Wait-free vs Lock-free comparison for Skip List

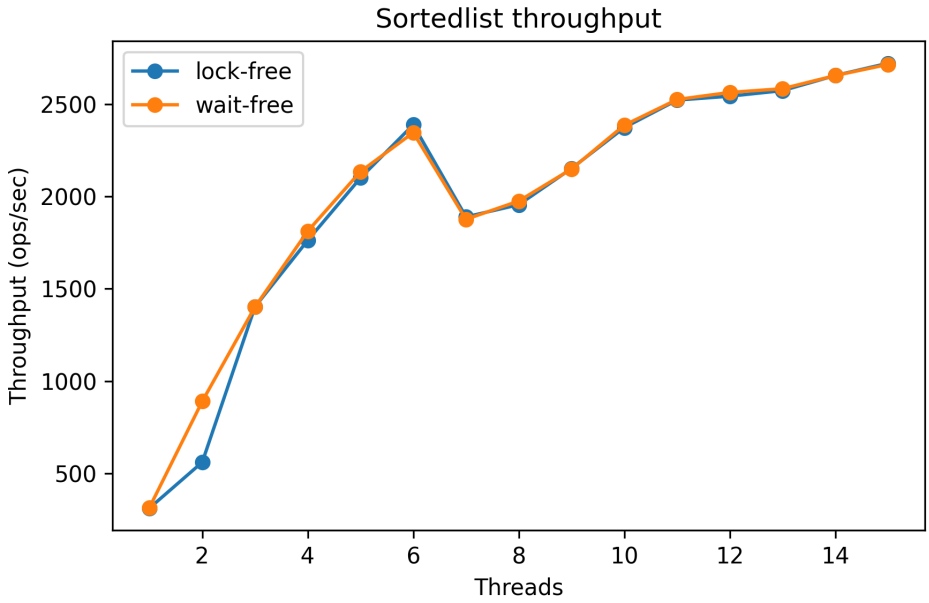

Figure 5: Wait-free vs Lock-free comparison for Sorted List

On measuring the cost of helping, we found that in the wait-free universal constructions, the threads seldom execute their own tasks, as shown in Table 1. Indeed, this matched our previous expectations.

Structure	Threads	Helped	Own	Help Ratio
stack	15	199,900,000	200,000	0.999000
queue	15	199,900,000	200,000	0.999000
bst	15	199,900,000	200,000	0.999000
sortedlist	15	199,900,000	200,000	0.999000
skiplist	15	199,900,000	200,000	0.999000

Table 1: Cost of helping.

5 Reflection on the Use of LLMs

Tools and models used. We used GitHub Copilot, Claude Sonnet 4.6, and a little ChatGPT-5. GitHub Copilot was used for generating the benchmarks, debugging the type errors, and generating the python scripts used for plotting. Claude Sonnet 4.6 was used mainly for syntax fixes and type errors, primarily in the stm-tests, since our state machine was a little convoluted. ChatGPT-5 was used primarily to summarize the explanations in the book.

What worked well. GitHub Copilot successfully generated the benchmarking files and the python scripts used to plot without any errors. Before running the TSAN, we asked it to check for potential data races, and it did not find any. Later, when I ran a thread sanitizer, it did not find any data race. It was specifically helpful in debugging the type errors related to `invoc` in our code. The Claude Sonnet 4.6 backup was really useful when Copilot wasn't able to fix a few bugs and started hallucinating, trying to justify what it did.

What did not work. Although Copilot was able to generate most scripts correctly, it struggled in handling type errors when writing modular OCaml code. It could not fix many stm-tests and made us use many attempts to get it right. Further, when we were using a hashtable for mapping Domain.self () to a unique number and were getting data races, it suggested locking these despite our implementations being non-blocking.

What was surprising. The surprising part was how well Copilot could gather context if given access to all the files in the repository. It was able to learn how QCheck-Lin and QCheck-STM were implemented and almost gave the correct code in our context.

What was difficult. Copilot faced issues in fixing type errors while using OCaml modules - significant prompting and involvement of Claude Sonnet 4.6 was also needed. Neither Copilot nor Claude Sonnet 4.6 could help debug the issue with Lin test because of repeating tids. They were trying to give some reason to show where our implementation was wrong, but that was usually not the case.

Your overall assessment. GitHub Copilot was extremely useful in this project. Backed up by some corrections from Claude Sonnet 4.6 and human-involvement, it greatly helped us with debugging the code and generating the parts of code that are straightforward, therefore improving our efficiency.

6 Conclusions

Universal constructions have a significant performance impact. They sacrifice a lot of throughput, as depicted in Figures 1 and 2. Though the wait-free variant's helping mechanism introduces overhead, it is offset by other reasons, as described in this report. Thus, the lock-free variant only has a small advantage over the wait-free variant. But the objective of the project was never to implement something that can provide a speed up - it was to show that given a sequential object, our constructions help make it into a wait-free, linearizable concurrent object. However, further studies have been done to make an efficient wait-free implementation which relies on selective help based on contention [1], so we could possibly implement an OCaml 5 implementation for the same in the future.

Contributions

Member	**Contribution (%)**	**Main responsibilities**
Venkat Koundinya Singaraju	33.33%	CAS Consensus, lock-free universal construction, purpose-built lock-free and mutex-protected sequential implementations, report and benchmarking code
Ventrapragada Krishna Sameer	33.33%	Wait-free universal construction, stm-tests for both the constructions, debugging issues in lin and stm-tests, final presentation.
Siddharth S George	33.33%	Implemented sequential objects (stack, queue, sorted list), debugged seq skiplist, manual and qcheck-lin tests, used tsan to detect data races
Total	**100%**	

survey

References

[1] Pascal Felber Andreia Correia, Pedro Ramalhete. A wait-free universal construct for large objects. 2019.

[2] M. Herlihy. Wait-free synchronization. *ACM TOPLAS*, 1991.

[3] Maurice Herlihy, Nir Shavit, Victor Luchangco, and Michael Spear. *The Art of Multiprocessor Programming*. Morgan Kaufmann, 2nd edition, 2020.